

✓ **Asignatura: Ciencia de Datos**

Unidad 2: Limpieza, Transformación y Calidad de los Datos - Teoría

Universidad Nacional de Chilecito

```
# 1ro debemos conectar el google colab
# 2do correr el siguiente código para poder montar la carpeta drive y poder tener acceso a las carpetas que tengo en mi Unidad

from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)

En Python, df es la abreviatura convencional para un DataFrame de Pandas, que es la estructura de datos fundamental de la biblioteca Pandas, diseñada para manipular y analizar datos tabulares bidimensionales (filas y columnas). Funciona similar a una hoja de cálculo o una tabla SQL, permitiendo gestionar datos heterogéneos. Aquí los detalles clave sobre df:

¿Qué es?: Un DataFrame de Pandas, un objeto tabular de dos dimensiones.

Convención: Aunque se puede llamar de cualquier forma, usar df es un estándar aceptado entre analistas y científicos de datos para identificar rápidamente un DataFrame.

Estructura: Se compone de filas etiquetadas y columnas que pueden contener diferentes tipos de datos (números, texto, booleanos).

✓ **Práctica N° 1**

Para el siguiente práctico vamos a utilizar el DataSet que lo podemos descargar desde ésta página:

<https://www.kaggle.com/datasets/cedricaubin/linkedin-data-analyst-jobs-listings?select=linkedin-jobs-usa.csv>

✓ **1° Paso: Cargamos el DataSet y Realizamos la Exploración Inicial**

```
# Con esta línea Importamos el DataSet con el que vamos a trabajar y lo asignamos a la variable pathCurso
pathCurso = '/content/drive/MyDrive/CienciaDatos/DataSet/linkedin-jobs-usa.csv'
```

1_1 Importamos las Librerías Necesarias

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib as plt

# Leemos el DataSet que está en pathCurso, pero ahora como un DataFrame

df_Trabajo = pd.read_csv(pathCurso)
```

```
# Con esta línea de código podemos saber cuantas Filas y Columnas tiene el DataSet
df_Trabajo.shape
```

(2845, 9)

```
# Con esta Línea podemos ver las 5 primeras filas con sus correspondientes columnas
df_Trabajo.head()
```

	title	company	description	onsite_remote	salary	location	criteria	posted_date	link
0	Data Analyst - Recent Graduate	PayPal	At PayPal (NASDAQ: PYPL), we believe that ever...	onsite	NaN	Buffalo-Niagara Falls Area	[{'Seniority level': 'Not Applicable'}, {'Empl...	2022-11-22	https://www.linkedin.com/jobs/view/data-analyst-recent-graduate-paypal
1	Data Analyst - Recent Graduate	PayPal	At PayPal (NASDAQ: PYPL), we believe that ever...	onsite	NaN	San Jose, CA	[{'Seniority level': 'Not Applicable'}, {'Empl...	2022-11-22	https://www.linkedin.com/jobs/view/data-analyst-recent-graduate-paypal
	Data Analyst - Recent Graduate	PayPal	At PayPal (NASDAQ: PYPL), we believe that ever...	onsite	NaN	Texas, United States	[{'Seniority level': 'Not Applicable'}, {'Empl...	2022-11-22	https://www.linkedin.com/jobs/view/data-analyst-recent-graduate-paypal

Con esta línea podremos ver la cantidad de columnas y los tipos de datos que no son null, la cantidad de memoria utilizada y el tipo de índice de la columna.

```
df_Trabajo.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2845 entries, 0 to 2844
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0    title           2845 non-null   object
1    company         2845 non-null   object
2    description      2845 non-null   object
3    onsite_remote    2845 non-null   object
4    salary          929 non-null    object
5    location        2845 non-null   object
6    criteria        2845 non-null   object
7    posted_date     2845 non-null   object
8    link            2845 non-null   object
dtypes: object(9)
memory usage: 200.2+ KB
```

2° Paso: Tratamientos de Datos Duplicados

Con esta línea vamos a eliminar las filas duplicadas que tenemos en el DataSet

```
df_Trabajo.drop_duplicates()
```

	title	company	description	onsite_remote	salary	location	criteria	posted_date	
0	Data Analyst - Recent Graduate	PayPal	At PayPal (NASDAQ: PYPL), we believe that ever...	onsite	NaN	Buffalo-Niagara Falls Area	[{'Seniority level': 'Not Applicable'}, {'Empl...	2022-11-22	https://www.linke
1	Data Analyst - Recent Graduate	PayPal	At PayPal (NASDAQ: PYPL), we believe that ever...	onsite	NaN	San Jose, CA	[{'Seniority level': 'Not Applicable'}, {'Empl...	2022-11-22	https://www.linke
2	Data Analyst	PayPal	At PayPal (NASDAQ: PYPL), we believe that ever...	onsite	NaN	Texas, United States	[{'Seniority level': 'Not Applicable'}, {'Empl...	2022-11-17	https://www.linke
3	Data Analyst	PayPal	At PayPal (NASDAQ: PYPL), we believe that ever...	onsite	NaN	Illinois, United States	[{'Seniority level': 'Not Applicable'}, {'Empl...	2022-11-17	https://www.linke
4	Entry-Level Data Analyst	The Federal Savings Bank	The Federal Savings Bank, a national bank and ...	onsite	NaN	Chicago, IL	[{'Seniority level': 'Entry level'}, {'Employment...	2022-11-17	https://www.linke
...	...	...	...	...	...	...	...	...	...

```
df_Trabajo.info()
```

3° Paso: Tratamientos de Datos Nulos

3_1 Eliminación de filas que contienen valores nulos

```
# Con esta línea vamos a Eliminar las filas que contienen al menos un valor nulo
df_sin_nulos = df_Trabajo.dropna()
```

```
# Con esta línea podemos saber cuantas Filas quedaron que NO tienen valores nulos
df_sin_nulos.shape
```

```
(929, 9)
```

3_2 Eliminación de columnas que contienen valores nulos

```
# Con esta línea vamos a Eliminar las columnas que contienen al menos un valor nulo
df_sin_nulos_columnas = df_Trabajo.dropna(axis=1)
```

```
# Con esta línea podemos saber cuantas Columnas quedaron que NO tienen valores nulos
df_sin_nulos_columnas.shape
```

4° Paso: Transformación de los Datos

4_1 Reemplazar los valores nulos NaNs por el valor de la Media

```
df_sin_nulos['salary']
```

	salary
8	\$100,000.00\r\n -\r\n \$1...
24	\$50,000.00\r\n -\r\n \$55...
35	\$100,000.00\r\n -\r\n \$1...
61	\$100,000.00\r\n -\r\n \$1...
88	\$100,000.00\r\n -\r\n \$1...
...	...
2834	\$30.00\r\n -\r\n \$33.00
2836	\$120,000.00\r\n -\r\n \$1...
2841	\$75,000.00\r\n -\r\n \$95...
2843	\$85,000.00\r\n -\r\n \$95...
2844	\$130,000.00\r\n -\r\n \$1...

929 rows × 1 columns

dtype: object

```
# Con esta línea los que hacemos es tomar la columna 'salary' que contiene los valores del salario en formato de cadena,
# eliminamos cualquier caracter que no sea numérico, extraemos los dígitos y convertimos la columna resultante en valores r
# El valor resultante se almacena en una nueva columna llamada 'salary_numeric'.
# Esto es muy útil cuando la columna 'salary' inicial contiene caracteres no numéricos y se desea trabajar con los salarios:
```

```
df_sin_nulos['salary_numeric'] = df_sin_nulos['salary'].replace('[^\d.]', '', regex=True).str.extract('(\d+)').astype(float)
```

```
<>:6: SyntaxWarning: invalid escape sequence '\d'
<>:6: SyntaxWarning: invalid escape sequence '\d'
<>:6: SyntaxWarning: invalid escape sequence '\d'
<>:6: SyntaxWarning: invalid escape sequence '\d'
/tmp/ipykernel_528/1962743071.py:6: SyntaxWarning: invalid escape sequence '\d'
  df_sin_nulos['salary_numeric'] = df_sin_nulos['salary'].replace('[^\d.]', '', regex=True).str.extract('(\d+)').astype(float)
/tmp/ipykernel_528/1962743071.py:6: SyntaxWarning: invalid escape sequence '\d'
  df_sin_nulos['salary_numeric'] = df_sin_nulos['salary'].replace('[^\d.]', '', regex=True).str.extract('(\d+)').astype(float)
/tmp/ipykernel_528/1962743071.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view
df_sin_nulos['salary_numeric'] = df_sin_nulos['salary'].replace('[^\d.]', '', regex=True).str.extract('(\d+)').astype(float)
```

```
df_sin_nulos['salary_numeric']
```

	salary_numeric
8	100000.0
24	50000.0
35	100000.0
61	100000.0
88	100000.0
...	...
2834	30.0
2836	120000.0
2841	75000.0
2843	85000.0
2844	130000.0

929 rows × 1 columns

dtype: float64

```
# Generamos una descripción en forma estadística
df_sin_nulos['salary_numeric'].describe()
```

	salary_numeric
count	929.000000
mean	47949.630786
std	47330.459576
min	22.000000
25%	40.000000
50%	60000.000000
75%	85000.000000
max	130000.000000

dtype: float64

```
df_Trabajo.describe()
```

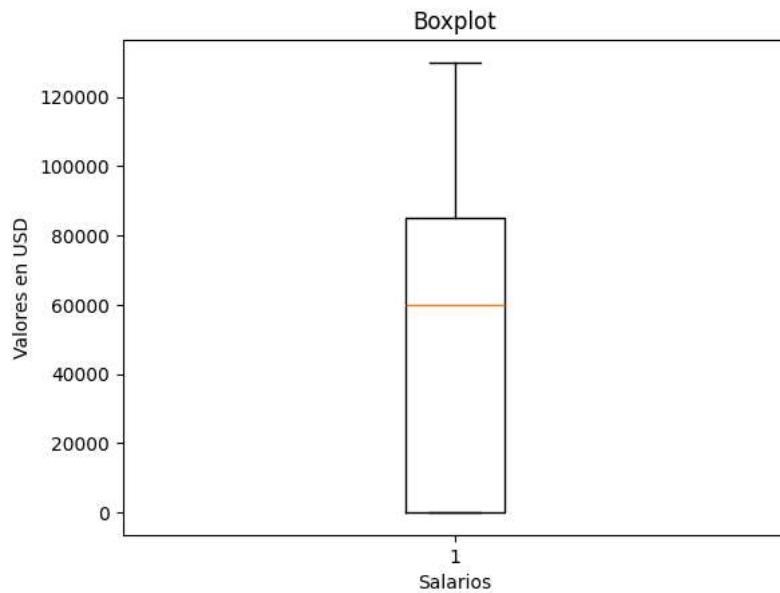
5° Paso: Vemos el DataFrame en Forma Gráfica

5_1 Boxplot

```
import matplotlib.pyplot as plt
plt.boxplot(df_sin_nulos['salary_numeric'])

# Le añadimos la etiqueta y el título
plt.xlabel('Salarios')
plt.ylabel('Valores en USD')
plt.title('Boxplot')

# Realizamos la representación gráfica
plt.show()
```

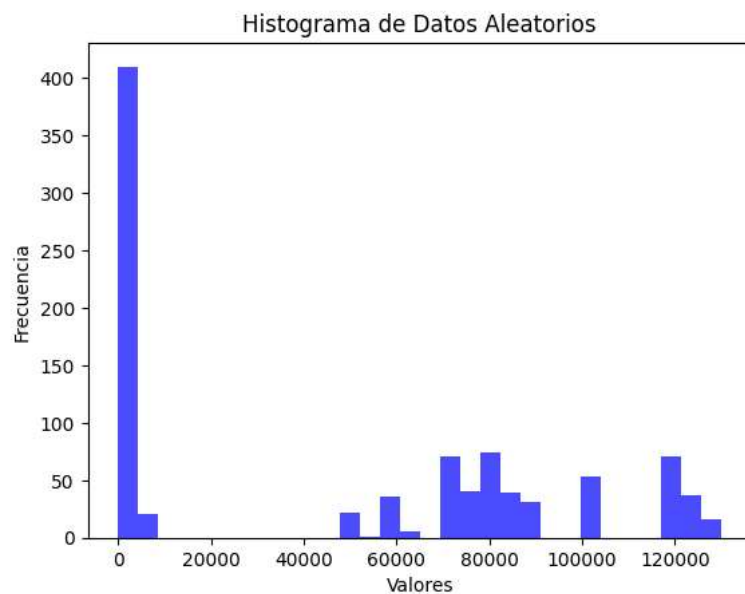


5_2 Histograma

```
plt.hist(df_sin_nulos['salary_numeric'], bins=30, color='blue', alpha=0.7)

# Le añadimos la etiqueta y el título
plt.xlabel('Valores')
plt.ylabel('Frecuencia')
plt.title('Histograma de Datos Aleatorios')

# Realizamos la representación gráfica
plt.show()
```

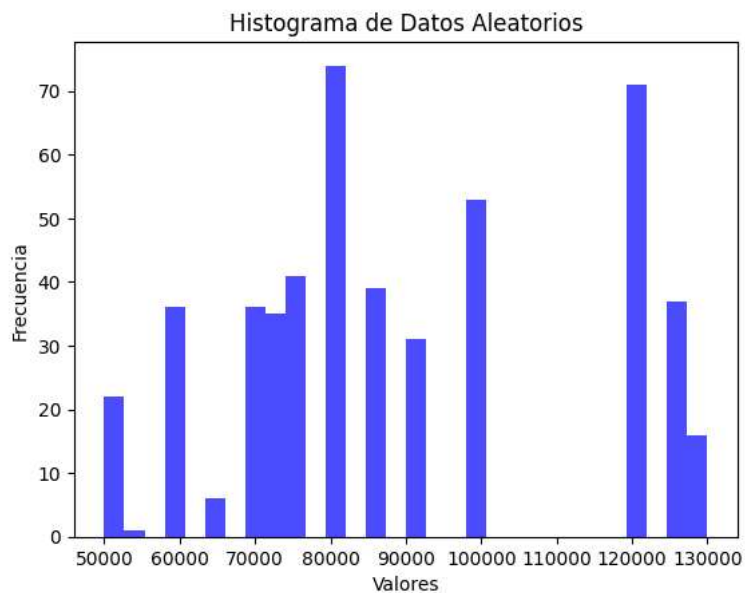


```
# Eliminamos las filas que contengan un salario mayor o igual a 20000
df_sin_nulos = df_sin_nulos[df_sin_nulos['salary_numeric'] >= 50000]
```

```
plt.hist(df_sin_nulos['salary_numeric'], bins=30, color='blue', alpha=0.7)

# Le añadimos la etiqueta y el título
plt.xlabel('Valores')
plt.ylabel('Frecuencia')
plt.title('Histograma de Datos Aleatorios')

# Realizamos la representación gráfica
plt.show()
```



6° Paso: Reemplazamos los Valores Nulos con la Media

6_1 Rellenamos

```
# Reemplazamos los valores nulos con un valor específico, en este caso el promedio
```

```
df_con_reemplazo = df_sin_nulos.fillna(df_sin_nulos['salary_numeric'].mean())
```

df_con_reemplazo

	title	company	description	onsite_remote	salary	location	criteria	posted_date	
8	Data Analyst	London Approach	The ideal candidate for the Data Analyst/Finan...	onsite	\$100,000.00\r\n-\r\n \$1...	Franklin, TN	[{'Seniority level': 'Associate'}, {'Employment...	2022-11-21	https://www.linke
24	Data Analyst	Eva Garland Consulting, LLC	Reporting to the Director of Operations, the D...	onsite	\$50,000.00\r\n-\r\n \$55...	Raleigh, NC	[{'Seniority level': 'Entry level'}, {'Employment...	2022-11-22	https://www.linke
35	Data Analyst	London Approach	The ideal candidate for the Data Analyst/Finan...	onsite	\$100,000.00\r\n-\r\n \$1...	Franklin, TN	[{'Seniority level': 'Associate'}, {'Employment...	2022-11-21	https://www.linke
61	Data Analyst	London Approach	The ideal candidate for the Data Analyst/Finan...	onsite	\$100,000.00\r\n-\r\n \$1...	Franklin, TN	[{'Seniority level': 'Associate'}, {'Employment...	2022-11-21	https://www.linke
88	Data Analyst	London Approach	The ideal candidate for the Data Analyst/Finan...	onsite	\$100,000.00\r\n-\r\n \$1...	Franklin, TN	[{'Seniority level': 'Associate'}, {'Employment...	2022-11-21	https://www.linke
...	...	...	...	...	...	...	...	...	...
2826	Data Analyst	IKEA	WHY WE WILL LOVE YOUWe set the digital agenda ... Odides IT is	hybrid	\$73,036.00\r\n-\r\n \$10...	Conshohocken, PA	[{'Seniority level': 'Not Applicable'}, {'Empl...	2022-10-27	https://www.linke

```
df_con_reemplazo.drop('salary', axis=1, inplace=True)
```

```
df_con_reemplazo
```

	title	company	description	onsite_remote	location	criteria	posted_date	
8	Data Analyst	London Approach	The ideal candidate for the Data Analyst/Finan...	onsite	Franklin, TN	{'Seniority level': 'Associate'}, {'Employment...	2022-11-21	https://www.linkedin.com/jobs/view/an
24	Data Analyst	Eva Garland Consulting, LLC	Reporting to the Director of Operations, the D...	onsite	Raleigh, NC	{'Seniority level': 'Entry level'}, {'Employment...	2022-11-22	https://www.linkedin.com/jobs/view/an
35	Data Analyst	London Approach	The ideal candidate for the Data Analyst/Finan...	onsite	Franklin, TN	{'Seniority level': 'Associate'}, {'Employment...	2022-11-21	https://www.linkedin.com/jobs/view/an
61	Data Analyst	London Approach	The ideal candidate for the Data Analyst/Finan...	onsite	Franklin, TN	{'Seniority level': 'Associate'}, {'Employment...	2022-11-21	https://www.linkedin.com/jobs/view/an
88	Data Analyst	London Approach	The ideal candidate for the Data Analyst/Finan...	onsite	Franklin, TN	{'Seniority level': 'Associate'}, {'Employment...	2022-11-21	https://www.linkedin.com/jobs/view/an
...	...	...	...	...	...	...	...	...
2826	Data Analyst	IKEA	WHY WE WILL LOVE YOUWe set the digital agenda ... Guiding IT is	hybrid	Conshohocken, PA	{'Seniority level': 'Not Applicable'}, {'Empl...	2022-10-27	https://www.linkedin.com/jobs/view/an

7° Paso: Agrupamos y Agregamos Datos

```
# Continuando con el ejemplo tomamos las columnas 'title' y 'salary'.
# Vamos a agrupar por título y calcular el promedio y la mediana del salario para cada título.

df_grouped = df_con_reemplazo.groupby('title')['salary_numeric'].agg(['mean', 'median', 'count']).reset_index()

# Renombramos las columnas para que se entienda mejor
df_grouped.columns = ['title', 'salary_mean', 'salary_median', 'job_count']

# Finalmente mostramos el nuevo DataFrame resultante
df_grouped
```


	title	salary_mean	salary_median	job_count
0	Analyst - Data Visualization	80000.00000	80000.0	35
1	Data Analyst	90894.12037	85000.0	432
2	Data Analyst (Hybrid)	80000.00000	80000.0	1
3	Data Analyst (SQL)	73800.00000	75000.0	25
4	Data Analyst - Energy	80000.00000	80000.0	1
5	Data Analyst/ \$100M Valuation/ Hybrid	120000.00000	120000.0	1
6	Data Analyst/Collector	55000.00000	55000.0	1
7	REMOTE Data Analyst (SQL, Operations)	80000.00000	80000.0	2

```
# Sacamos la Media
df_grouped['salary_mean'] = df_grouped['salary_mean'].round(0).astype(int)

# Sacamos la Mediana
df_grouped['salary_median'] = df_grouped['salary_median'].round(0).astype(int)
```

```
# Mostramos el nuevo DataSet
df_grouped
```

	title	salary_mean	salary_median	job_count
0	Analyst - Data Visualization	80000	80000	35
1	Data Analyst	90894	85000	432
2	Data Analyst (Hybrid)	80000	80000	1
3	Data Analyst (SQL)	73800	75000	25
4	Data Analyst - Energy	80000	80000	1
5	Data Analyst/ \$100M Valuation/ Hybrid	120000	120000	1
6	Data Analyst/Collector	55000	55000	1
7	REMOTE Data Analyst (SQL, Operations)	80000	80000	2

8° Paso: Exportamos el CSV Resultante

```
# Exportamos el nuevo archivo csv
# Debemos especificar la ruta y colocar el nombre al archivo csv
output_path_reemplazo = '/content/drive/MyDrive/CienciaDatos/nuevo_linkedin_jobs_usa.csv'

# Con esta línea Exportamos el DataFrame a un archivo csv
df_con_reemplazo.to_csv(output_path_reemplazo, index=False)
```

Práctica N° 2

Para el siguiente práctico vamos a utilizar el DataSet que lo podemos descargar desde ésta página:

<https://www.kaggle.com/datasets/likithagedipudi/job-board-effectiveness-for-college-students>

1° Cargamos el DataSet y Realizamos la Exploración Inicial

```
# Con esta línea asignamos el DataSet a la variable pathCurso
pathCurso = '/content/drive/MyDrive/CienciaDatos/DataSet/job_search_platform_efficacy_100k.csv'
```

```
import numpy as np
import pandas as pd
#import matplotlib.pyplot as plt

# Leemos el DataSet que está en pathCurso, pero ahora como un DataFrama

df_Lavoral = pd.read_csv(pathCurso)
```

```
# Que nos muestra: Total de filas y columnas - Nombres de columnas y tipos de datos - Valores no nulos - Uso de memoria
df_Lavoral.info()

#print(df_Lavoral.info())
```

```
# Generamos una descripción en forma estadística
df_Lavoral.describe()

#print(df_Lavoral.describe(include='all'))
```

```
# Permite realizar el calcula y muestra el número total de valores nulos o faltantes (NaN, None) por cada columna en un Dat
df_Lavoral.isnull()

#df_Lavoral.isnull().sum()

# print(df_Lavoral.isnull().sum())
```

Haz doble clic (o ingresa) para editar

▼ 2° Paso: Identificación de Datos Inconsistentes y Outliers

```
# Buscamos datos inconsistentes, por ejemplo valores que esten por fuera del rango o en categorías invalidas, que serían va
for col in df_Lavoral.select_dtypes(include='object').columns:
    print(f"{col}: {df_Lavoral[col].unique()[:10]}")
```

```
# Ahora vamos a detectar valores lógicos que están fuera de rango
datos_inconsistentes = df_Lavoral[
    (df_Lavoral['Role_Relevance'] < 0) | (df_Lavoral['Role_Relevance'] > 10)
]

print("Registros con datos inconsistentes:", len(datos_inconsistentes))
```

```
# detección de datos Outliers (IQR)
def detectar_outliers_iqr(df_Lavoral, col):
    Q1 = df_Lavoral[col].quantile(0.25)
    Q3 = df_Lavoral[col].quantile(0.75)
    IQR = Q3 - Q1
    return df_Lavoral[(df_Lavoral[col] < Q1 - 1.5 * IQR) | (df_Lavoral[col] > Q3 + 1.5 * IQR)]

outliers_role = detectar_outliers_iqr(df_Lavoral, 'Role_Relevance')
print("Outliers en Role_Relevance:", len(outliers_role))
```

▼ 3° Paso: Manejo de Datos Faltantes

```
# Lo primero que vamos a realizar es la Eliminación de las filas con demasiados datos nulos
df_drop = df_Lavoral.dropna(thresh=15)

#df_drop.info()

#df_drop.isnull()
```

```
# Vamos a utilizar la función mean() para obtener la media de las variables numéricas
df_Lavoral['Role_Relevance'] = df_Lavoral['Role_Relevance'].fillna(df_Lavoral['Role_Relevance'].mean())
```

```
# Vamos a utilizar la función mode() para obtener la moda de las variables categóricas
df_Lavoral['University_Rating'] = df_Lavoral['University_Rating'].fillna(df_Lavoral['University_Rating'].mode()[0])
```

```
# Ahora vamos a realizar Imputaciones múltiples (MICE)

# En primersa instancia debemos importar las librerías necesarias
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

num_cols = df_Lavoral.select_dtypes(include=np.number).columns

imputer = IterativeImputer(random_state=0)
df_Lavoral[num_cols] = imputer.fit_transform(df_Lavoral[num_cols])
```

4° Paso: Manejo de Datos Duplicados y Errores de Codificación

```
# Detección y Eliminación de datos duplicados
duplicados = df_Lavoral[df_Lavoral.duplicated()]
print("Duplicados:", len(duplicados))
```

```
df_Lavoral = df_Lavoral.drop_duplicates()
```

```
# Ahora vamos a realizar una normalización del texto
df_Lavoral['University_Rating'] = df_Lavoral['University_Rating'].str.strip().str.lower()
```

```
# Ahora vamos a realizar una corrección de categorías
df_Lavoral['University_Rating'] = df_Lavoral['University_Rating'].replace({
    'mid tier': 'mid-tier',
    'lower tier': 'lower-tier'
})
```

5° Paso: Mejoras en la Calidad de los Datos

```
# Conversión de tipos
df_Lavoral['Accepted_Offer'] = df_Lavoral['Accepted_Offer'].astype('int')

# Creación de variables derivadas
df_Lavoral['High_Relevance'] = df_Lavoral['Role_Relevance'] > 7

# Validación final
print(df_Lavoral.isnull().sum())
```

6° Paso: Los Riesgos que Puedo Tener

- * Impuntuación Incorrecta.
- * Elemento de la lista
- * Outliers no tratados
- * Errores de codificación
- * Datos duplicados

7° Paso: Guardar el Nuevo DataSet

```
# Guardamos el nuevo data set pero ya procesado

df_Lavoral.to_csv("dataset_limpio.csv", index=False)
```

Práctica N° 3

Para el siguiente práctico vamos a utilizar el DataSet que lo podemos descargar desde ésta página:

<https://www.kaggle.com/datasets/moezalikhan/global-coffee-shop-sales-dataset>

```
# 1ro debemos conectar el google colab
# 2do correr el siguiente código para poder montar la carpeta drive y poder tener acceso a las carpetas que tengo en mi Uni

from google.colab import drive
drive.mount('/content/drive')
```

1° Cargamos el DataSet y Realizamos la Exploración Inicial

```
# Con esta línea asignamos el set de datos a la variable pathCurso

pathCurso = '/content/drive/MyDrive/CienciaDatos/DataSet/coffee_shop_sales.csv'
```

```
# Importamos las librerías necesarias para poder trabajar
import numpy as np
import pandas as pd

# Leemos el DataSet que está en pathCurso, pero ahora como un DataFrame

df_Coffe_Shop = pd.read_csv(pathCurso)

# A partir de ahora se llama "df_CoffeShop"
```

```
# Permite realizar una vista rápida de las primeras 5 filas de una DataFrame
df_Coffe_Shop.head()
```

```
# Que nos muestra: Total de filas y columnas - Nombres de columnas y tipos de datos - Valores no nulos - Uso de memoria

df_Coffe_Shop.info()

#print(df_Lavoral.info())
```

```
# Generamos una descripción en forma estadística
df_Coffe_Shop.describe()

#print(df_Lavoral.describe(include='all'))
```

2° Paso: Identificación de Datos Inconsistentes y Outliers

```
# Buscamos datos inconsistentes, con valores negativos
datos_inconsistentes = df_Coffe_Shop[(df_Coffe_Shop['quantity'] < 0) | (df_Coffe_Shop['unit_price'] < 0)]
```

```
# Buscamos variables categorías que estén mal escritas
print(df_Coffe_Shop['city'].unique())
```

```
['Melbourne' 'Manchester' 'Toronto' 'Los Angeles' 'Vancouver' 'Chicago'
 'Sydney' 'New York' 'London']
```

```
# detección de datos Outliers (IQR)
def detectar_outliers(col):
    Q1 = df_Coffe_Shop[col].quantile(0.25)
    Q3 = df_Coffe_Shop[col].quantile(0.75)
    IQR = Q3 - Q1

    limite_inferior = Q1 - 1.5 * IQR
    limite_superior = Q3 + 1.5 * IQR

    return df_Coffe_Shop[(df_Coffe_Shop[col] < limite_inferior) | (df_Coffe_Shop[col] > limite_superior)]

outliers_qty = detectar_outliers('quantity')
outliers_price = detectar_outliers('unit_price')

print("Outliers en cantidad:\n", outliers_qty)
print("Outliers en precio:\n", outliers_price)
```

3° Paso: Manejo de Datos Faltantes

```
# Identificamos los datos faltantes
print(df_Coffe_Shop.isnull().sum())
```

```
# Ahora eliminamos esos datos faltantes
df_drop = df_Coffe_Shop.dropna()
```

```
# Ahora vamos a realizar Imputaciones simple
# Primero a los valores numéricos lo completamos con la media
df_Coffe_Shop['unit_price'].fillna(df_Coffe_Shop['unit_price'].mean(), inplace=True)

# Segundo a los valores categóricos con la moda
df_Coffe_Shop['store_location'].fillna(df_Coffe_Shop['store_location'].mode()[0], inplace=True)
```

```
# Ahora vamos a realizar Imputaciones múltiples

from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

num_cols = df_Coffe_Shop.select_dtypes(include=np.number)

imputer = IterativeImputer(random_state=0)
df_Coffe_Shop[num_cols.columns] = imputer.fit_transform(num_cols)
```

✓ 4° Paso: Manejo de Datos Duplicados y Errores de Codificación

```
# Detección y Eliminación de datos duplicados
datos_duplicados = df_Coffe_Shop[df_Coffe_Shop.duplicated()]
print("Duplicados:\n", datos_duplicados)

df_nuevo = df_Coffe_Shop.drop_duplicates()
```

```
# Eliminación de errores de codificación
```